SPHOORTHY ENGINEERING COLLEGE

An Autonomous Institution Approved by AICTE & Accredited with NAAC Grade 'A'
Department of Examination & Academic Affairs · Hyderabad — 501510

Data Structures & Algorithms (Unit 1 to 5 Complete Notes)

Program / Dept: B.Tech / MBA — CSE	**Course Code / Title:** Data Structures & Algorithms (CS301PC)
Academic Regulation: Autonomous (SR23 / SR24)	**Curriculum Semester:** Semester 3 (Year 2)
Credits / Contact Hours: 3 Credits · 48 Lecture Hours	**Evaluation Scheme:** Continuous Internal (40M) + End Sem (60M)

Course Overview & Pedagogical Objectives:

Comprehensive reference notes on Stacks, Queues, Linked Lists, Trees (BST, AVL, B-Trees), Graph traversals (BFS/DFS), and Hashing techniques with complexity analysis. The primary objective of this course is to equip students with sound theoretical rigor, analytical problem-solving competencies, and real-world system engineering capabilities demanded by premier technological organizations and competitive graduate benchmarks.

Course Outcomes (COs) & Bloom's Taxonomy Mapping:

CO ID	Course Learning Outcome Description	Bloom's Level
CO 1	Understand and formulate fundamental concepts of Unit 1: Linear Data Structures.	L2 Understand
CO 2	Analyze algorithms, mechanisms, and models in Unit 2: Queues and Recursion.	L4 Analyze
CO 3	Design and evaluate robust architectures for Unit 3: Non-Linear Structures: Trees.	L5 Evaluate
CO 4	Implement and optimize operational pipelines in Unit 4: Graph Algorithms.	L3 Apply
CO 5	Integrate state-of-the-art frameworks in Unit 5: Sorting & Hashing for industrial solutions.	L6 Create

Comprehensive Module Index:

Unit	Title & Core Concepts	Page
Unit 1	**Unit 1: Linear Data Structures**: Abstract Data Types, Dynamic Arrays, Singly & Doubly Linked Lists, Circular Lists, Stack implementations, and ...	Page 2
Unit 2	**Unit 2: Queues and Recursion**: Linear Queue, Circular Queue, Double-Ended Queue (Deque), Priority Queue with Binary Heaps, and Recursion stac...	Page 3
Unit 3	**Unit 3: Non-Linear Structures: Trees**: Binary Tree traversals, Binary Search Tree (BST) operations, Self-Balancing AVL trees with rotations (LL, RR, ...	Page 4
Unit 4	**Unit 4: Graph Algorithms**: Graph representation (Adjacency Matrix & List), Depth First Search (DFS), Breadth First Search (BFS), Minimum ...	Page 5
Unit 5	**Unit 5: Sorting & Hashing**: Merge Sort, Quick Sort, Radix Sort, Open Addressing (Linear Probing, Quadratic, Double Hashing), Separate Chai...	Page 6

UNIT 1: LINEAR DATA STRUCTURES • *Core Theory & Practice Guide*

1.1 Technical Exposition & Conceptual Foundations

The study of unit 1: linear data structures forms an indispensable cornerstone of Data Structures & Algorithms (CS301PC). Specifically, Abstract Data Types, Dynamic Arrays, Singly & Doubly Linked Lists, Circular Lists, Stack implementations, and Expression evaluation (Infix to Postfix). Modern applications require deep awareness of edge conditions, invariant properties, and trade-offs between computational throughput, latency overhead, memory footprint, and architectural scalability.

In university examinations and practical engineering systems, solutions must be formulated with mathematical clarity. Each sub-mechanism must satisfy safety, liveness, and fault-tolerance constraints while adhering to standard autonomous syllabus requirements.

1.2 Architectural Principles & Comparative Analysis

Component / Topic	Key Mechanism & Principle	Operational Trade-off
Theoretical Model	Formal representation of Unit 1 entities and lifecycle stages.	Higher mathematical abstraction vs implementation friction.
Algorithmic Engine	State machine transitions, recurrence guarantees, and verification steps.	Time complexity bounds vs memory consumption.
Real-World Systems	Deployment in enterprise distributed systems and contemporary toolchains.	Fault-tolerant redundancy vs network overhead.

1.3 Practical Formulation & Execution Workflow

```
// Standard Implementation Outline & Pseudocode for Unit 1
1. Initialize system data structures and state variables under initial constraints.
2. Execute continuous invariant checking across state boundaries.
3. Apply optimization bounds: O(log N) average time or amortized linear bounds.
4. Catch boundary failure cases (underflow, overflow, concurrency race, null references).
5. Commit verified state or invoke transactional rollback upon validation failure.
```

1.4 Autonomous End-Semester Examination Question Bank

Type	Question & Expected Technical Answer Outline	Marks
Part A	**Q1:** Define the primary objective of Unit 1 and state two essential real-world applications. • *Expected: Exact definition, mathematical constraint formulation, and dual industry examples.*	**2 M**
Part A	**Q2:** Differentiate between static vs dynamic formulations of key mechanisms in Unit 1. • *Expected: 2-column comparison table covering memory, allocation time, and failure recovery.*	**2 M**
Part B	**Q3:** Explain the complete working architecture of Unit 1 with neat functional block diagrams and step-by-step algorithmic trace. • *Expected: Labeled diagram (3M), mathematical derivation (3M), algorithm steps (2M), complexity analysis (2M).*	**10 M**
Part B	**Q4:** Analyze the worst-case, best-case, and average-case performance bounds for processes in Unit 1 with worked numerical example. • *Expected: Step-by-step problem solution, boundary checks, and conclusive optimality proof.*	**10 M**

UNIT 2: QUEUES AND RECURSION • *Core Theory & Practice Guide*

2.1 Technical Exposition & Conceptual Foundations

The study of unit 2: queues and recursion forms an indispensable cornerstone of Data Structures & Algorithms (CS301PC). Specifically, Linear Queue, Circular Queue, Double-Ended Queue (Deque), Priority Queue with Binary Heaps, and Recursion stack frame trace. Modern applications require deep awareness of edge conditions, invariant properties, and trade-offs between computational throughput, latency overhead, memory footprint, and architectural scalability.

In university examinations and practical engineering systems, solutions must be formulated with mathematical clarity. Each sub-mechanism must satisfy safety, liveness, and fault-tolerance constraints while adhering to standard autonomous syllabus requirements.

2.2 Architectural Principles & Comparative Analysis

Component / Topic	Key Mechanism & Principle	Operational Trade-off
Theoretical Model	Formal representation of Unit 2 entities and lifecycle stages.	Higher mathematical abstraction vs implementation friction.
Algorithmic Engine	State machine transitions, recurrence guarantees, and verification steps.	Time complexity bounds vs memory consumption.
Real-World Systems	Deployment in enterprise distributed systems and contemporary toolchains.	Fault-tolerant redundancy vs network overhead.

2.3 Practical Formulation & Execution Workflow

```
// Standard Implementation Outline & Pseudocode for Unit 2
1. Initialize system data structures and state variables under initial constraints.
2. Execute continuous invariant checking across state boundaries.
3. Apply optimization bounds: O(log N) average time or amortized linear bounds.
4. Catch boundary failure cases (underflow, overflow, concurrency race, null references).
5. Commit verified state or invoke transactional rollback upon validation failure.
```

2.4 Autonomous End-Semester Examination Question Bank

Type	Question & Expected Technical Answer Outline	Marks
Part A	**Q1:** Define the primary objective of Unit 2 and state two essential real-world applications. • *Expected: Exact definition, mathematical constraint formulation, and dual industry examples.*	**2 M**
Part A	**Q2:** Differentiate between static vs dynamic formulations of key mechanisms in Unit 2. • *Expected: 2-column comparison table covering memory, allocation time, and failure recovery.*	**2 M**
Part B	**Q3:** Explain the complete working architecture of Unit 2 with neat functional block diagrams and step-by-step algorithmic trace. • *Expected: Labeled diagram (3M), mathematical derivation (3M), algorithm steps (2M), complexity analysis (2M).*	**10 M**
Part B	**Q4:** Analyze the worst-case, best-case, and average-case performance bounds for processes in Unit 2 with worked numerical example. • *Expected: Step-by-step problem solution, boundary checks, and conclusive optimality proof.*	**10 M**

UNIT 3: NON-LINEAR STRUCTURES: TREES • *Core Theory & Practice Guide*

3.1 Technical Exposition & Conceptual Foundations

The study of unit 3: non-linear structures: trees forms an indispensable cornerstone of Data Structures & Algorithms (CS301PC). Specifically, Binary Tree traversals, Binary Search Tree (BST) operations, Self-Balancing AVL trees with rotations (LL, RR, LR, RL), and Red-Black Trees. Modern applications require deep awareness of edge conditions, invariant properties, and trade-offs between computational throughput, latency overhead, memory footprint, and architectural scalability.

In university examinations and practical engineering systems, solutions must be formulated with mathematical clarity. Each sub-mechanism must satisfy safety, liveness, and fault-tolerance constraints while adhering to standard autonomous syllabus requirements.

3.2 Architectural Principles & Comparative Analysis

Component / Topic	Key Mechanism & Principle	Operational Trade-off
Theoretical Model	Formal representation of Unit 3 entities and lifecycle stages.	Higher mathematical abstraction vs implementation friction.
Algorithmic Engine	State machine transitions, recurrence guarantees, and verification steps.	Time complexity bounds vs memory consumption.
Real-World Systems	Deployment in enterprise distributed systems and contemporary toolchains.	Fault-tolerant redundancy vs network overhead.

3.3 Practical Formulation & Execution Workflow

```
// Standard Implementation Outline & Pseudocode for Unit 3
1. Initialize system data structures and state variables under initial constraints.
2. Execute continuous invariant checking across state boundaries.
3. Apply optimization bounds: O(log N) average time or amortized linear bounds.
4. Catch boundary failure cases (underflow, overflow, concurrency race, null references).
5. Commit verified state or invoke transactional rollback upon validation failure.
```

3.4 Autonomous End-Semester Examination Question Bank

Type	Question & Expected Technical Answer Outline	Marks
Part A	**Q1:** Define the primary objective of Unit 3 and state two essential real-world applications. • *Expected: Exact definition, mathematical constraint formulation, and dual industry examples.*	**2 M**
Part A	**Q2:** Differentiate between static vs dynamic formulations of key mechanisms in Unit 3. • *Expected: 2-column comparison table covering memory, allocation time, and failure recovery.*	**2 M**
Part B	**Q3:** Explain the complete working architecture of Unit 3 with neat functional block diagrams and step-by-step algorithmic trace. • *Expected: Labeled diagram (3M), mathematical derivation (3M), algorithm steps (2M), complexity analysis (2M).*	**10 M**
Part B	**Q4:** Analyze the worst-case, best-case, and average-case performance bounds for processes in Unit 3 with worked numerical example. • *Expected: Step-by-step problem solution, boundary checks, and conclusive optimality proof.*	**10 M**

UNIT 4: GRAPH ALGORITHMS • *Core Theory & Practice Guide*

4.1 Technical Exposition & Conceptual Foundations

The study of unit 4: graph algorithms forms an indispensable cornerstone of Data Structures & Algorithms (CS301PC). Specifically, Graph representation (Adjacency Matrix & List), Depth First Search (DFS), Breadth First Search (BFS), Minimum Spanning Tree (Prim's & Kruskal's), and Dijkstra's Shortest Path. Modern applications require deep awareness of edge conditions, invariant properties, and trade-offs between computational throughput, latency overhead, memory footprint, and architectural scalability.

In university examinations and practical engineering systems, solutions must be formulated with mathematical clarity. Each sub-mechanism must satisfy safety, liveness, and fault-tolerance constraints while adhering to standard autonomous syllabus requirements.

4.2 Architectural Principles & Comparative Analysis

Component / Topic	Key Mechanism & Principle	Operational Trade-off
Theoretical Model	Formal representation of Unit 4 entities and lifecycle stages.	Higher mathematical abstraction vs implementation friction.
Algorithmic Engine	State machine transitions, recurrence guarantees, and verification steps.	Time complexity bounds vs memory consumption.
Real-World Systems	Deployment in enterprise distributed systems and contemporary toolchains.	Fault-tolerant redundancy vs network overhead.

4.3 Practical Formulation & Execution Workflow

```
// Standard Implementation Outline & Pseudocode for Unit 4
1. Initialize system data structures and state variables under initial constraints.
2. Execute continuous invariant checking across state boundaries.
3. Apply optimization bounds: O(log N) average time or amortized linear bounds.
4. Catch boundary failure cases (underflow, overflow, concurrency race, null references).
5. Commit verified state or invoke transactional rollback upon validation failure.
```

4.4 Autonomous End-Semester Examination Question Bank

Type	Question & Expected Technical Answer Outline	Marks
Part A	**Q1:** Define the primary objective of Unit 4 and state two essential real-world applications. • *Expected: Exact definition, mathematical constraint formulation, and dual industry examples.*	**2 M**
Part A	**Q2:** Differentiate between static vs dynamic formulations of key mechanisms in Unit 4. • *Expected: 2-column comparison table covering memory, allocation time, and failure recovery.*	**2 M**
Part B	**Q3:** Explain the complete working architecture of Unit 4 with neat functional block diagrams and step-by-step algorithmic trace. • *Expected: Labeled diagram (3M), mathematical derivation (3M), algorithm steps (2M), complexity analysis (2M).*	**10 M**
Part B	**Q4:** Analyze the worst-case, best-case, and average-case performance bounds for processes in Unit 4 with worked numerical example. • *Expected: Step-by-step problem solution, boundary checks, and conclusive optimality proof.*	**10 M**

UNIT 5: SORTING & HASHING • *Core Theory & Practice Guide*

5.1 Technical Exposition & Conceptual Foundations

The study of unit 5: sorting & hashing forms an indispensable cornerstone of Data Structures & Algorithms (CS301PC). Specifically, Merge Sort, Quick Sort, Radix Sort, Open Addressing (Linear Probing, Quadratic, Double Hashing), Separate Chaining, and Collision Resolution. Modern applications require deep awareness of edge conditions, invariant properties, and trade-offs between computational throughput, latency overhead, memory footprint, and architectural scalability.

In university examinations and practical engineering systems, solutions must be formulated with mathematical clarity. Each sub-mechanism must satisfy safety, liveness, and fault-tolerance constraints while adhering to standard autonomous syllabus requirements.

5.2 Architectural Principles & Comparative Analysis

Component / Topic	Key Mechanism & Principle	Operational Trade-off
Theoretical Model	Formal representation of Unit 5 entities and lifecycle stages.	Higher mathematical abstraction vs implementation friction.
Algorithmic Engine	State machine transitions, recurrence guarantees, and verification steps.	Time complexity bounds vs memory consumption.
Real-World Systems	Deployment in enterprise distributed systems and contemporary toolchains.	Fault-tolerant redundancy vs network overhead.

5.3 Practical Formulation & Execution Workflow

```
// Standard Implementation Outline & Pseudocode for Unit 5
1. Initialize system data structures and state variables under initial constraints.
2. Execute continuous invariant checking across state boundaries.
3. Apply optimization bounds: O(log N) average time or amortized linear bounds.
4. Catch boundary failure cases (underflow, overflow, concurrency race, null references).
5. Commit verified state or invoke transactional rollback upon validation failure.
```

5.4 Autonomous End-Semester Examination Question Bank

Type	Question & Expected Technical Answer Outline	Marks
Part A	**Q1:** Define the primary objective of Unit 5 and state two essential real-world applications. • *Expected: Exact definition, mathematical constraint formulation, and dual industry examples.*	**2 M**
Part A	**Q2:** Differentiate between static vs dynamic formulations of key mechanisms in Unit 5. • *Expected: 2-column comparison table covering memory, allocation time, and failure recovery.*	**2 M**
Part B	**Q3:** Explain the complete working architecture of Unit 5 with neat functional block diagrams and step-by-step algorithmic trace. • *Expected: Labeled diagram (3M), mathematical derivation (3M), algorithm steps (2M), complexity analysis (2M).*	**10 M**
Part B	**Q4:** Analyze the worst-case, best-case, and average-case performance bounds for processes in Unit 5 with worked numerical example. • *Expected: Step-by-step problem solution, boundary checks, and conclusive optimality proof.*	**10 M**

Standard Recommended Textbooks & Digital References:

1. *Primary Curriculum Standard Textbook* — Pearson Higher Education / McGraw-Hill Education (Latest Edition).
2. *Autonomous Reference Syllabus Blueprint* — Sphoorthy Engineering College Academic Council.

3. *NPTEL & IEEE Computer Society Guidelines* — Open Educational Resources & Peer-reviewed Publications.